Algoritmos y Estructuras de Datos
Trabajo Práctico 4 - 2026
Implementación de un Sistema Simple de Gestión de Tratamientos Médicos (v4.0)

a.) Introducción

Se pedirá desarrollar una cuarta versión (la versión 4.0) del mismo sistema simple que se planteó para el TP1, el TP2 y el TP3 (gestión de tratamientos médicos en un centro de salud) en base al mismo esquema de identificación de las enfermedades, problemas de salud y causas de muerte del modelo **ICD10** (*International Classification of Diseases*, versión 10) ya usado en los TPs anteriores (vea los enunciados del TP1, TP2 y TP3 para recordar los detalles, que deberá volver a tener en cuenta en este nuevo trabajo).

En esta nueva versión, el centro de salud quiere saber qué parte de lo facturado por cada tratamiento es **cubierta por la obra social** del paciente, y qué parte queda **a cargo del propio paciente**. Para eso, además del archivo de tratamientos que ya conocen del TP3, se dispondrá de un segundo archivo con el **padrón de afiliados**, en el que se indica para cada paciente el nivel de cobertura de su obra social y el tope (límite) que esa cobertura tiene en cada atención.

Importante: la comprensión de este texto en su totalidad es parte de lo que se evalúa, esto implica que no respetar los formatos esperados para las entradas y salidas o los resultados esperados será penalizado con baja de puntos o incluso aplazo. Se les recomienda encarecidamente leer TODO el enunciado antes de sentarse a desarrollar una solución.

b.) Entradas

En esta versión deberán procesar **dos archivos** de texto en formato CSV (valores separados por comas). En ambos archivos, la primera línea es una línea especial que indica los nombres de los campos que contiene cada registro, y debe ser ignorada.

Archivo 1: coberturas.csv (padrón de afiliados)

Contiene una línea por cada afiliado (o sea, por cada paciente que tiene obra social). No hay dos líneas con el mismo DNI, y las líneas **NO vienen ordenadas** de ninguna forma en particular. Cada línea del archivo contiene los siguientes campos (en el orden que se detalla a continuación):

1. **DNI del afiliado** (sin puntos): número entero representando el DNI del afiliado.
2. **Código de obra social**: número entero entre 1 y 20 (ambos incluidos) que identifica a la obra social a la que pertenece el afiliado.
3. **Nivel de cobertura**: número entero entre 1 y 3 (ambos incluidos) que representa el plan contratado por el afiliado. El nivel determina qué porcentaje del monto final de cada tratamiento cubre la obra social (vea la tabla de porcentajes de cobertura más adelante).

4. **Tope por atención**: número flotante que representa el monto máximo que la obra social está dispuesta a cubrir en cada tratamiento de ese afiliado (sin importar cuántos tratamientos tenga).

Ejemplo: en el siguiente modelo mostramos qué forma podría tener un pequeño archivo coberturas.csv:

```
DNI del Afiliado,Codigo Obra Social,Nivel de cobertura,Tope por atencion
17994000,17,3,133200.07
16887842,20,3,80473.79
48424442,19,2,58792.09
51218551,10,1,83958.58
```

Archivo 2: tratamientos.csv

Tiene **exactamente el mismo formato** que el archivo tratamientos.csv del TP3 (una línea por cada tratamiento con los campos DNI del paciente, nombre, apellido, código ICD10, monto base, complejidad e identificador de algoritmo de cálculo del monto final). Vuelva a leer el enunciado del TP3 para recordar el significado de cada campo.

Tenga en cuenta que un mismo paciente puede aparecer en muchos tratamientos (en cualquier orden), y que puede haber tratamientos de pacientes cuyo DNI **no figura** en el padrón de afiliados (es decir, pacientes que no tienen obra social). Ejemplo:

```
DNI del Paciente,Nombre del paciente,Apellido del paciente,Codigo ICD10,Monto
base,Complejidad,ID_Algoritmo
48187197,Lucas,Garcia,A02.0,69098.45,A,2
24984372,Valentina,Mendez,M18.7,69088.08,R,11
17994000,Jorge,Acosta,S23.4,25321.90,A,1
24984372,Valentina,Mendez,C16.6,83422.24,R,3
```

El programa será ejecutado y evaluado mediante un programa de control (o *runner*), y ese runner espera que los datos se carguen desde los archivos de texto exclusivamente. Por lo tanto, insistimos fuertemente en eso: el programa **no debe tomar datos desde el teclado** (salvo para el ingreso de la opción seleccionada del menú). Tampoco debe haber en ninguna parte nada que solicite al usuario presionar una tecla para continuar.

Es **MUY IMPORTANTE** que esos archivos se llamen **coberturas.csv** y **tratamientos.csv** y que estén almacenados en la misma carpeta del proyecto que contiene al programa fuente que desarrollen. Si están haciendo pruebas con otros archivos, no hay problemas, pero ASEGÚRENSE de que cuando entreguen el programa para su calificación, la función open() esté abriendo exclusivamente archivos llamados coberturas.csv y tratamientos.csv Y NO OTROS.

c.) Cálculo del monto final y de la cobertura

El **monto final** de cada tratamiento se calcula **exactamente igual que en el TP3 Base (SIN incluir los cálculos para los enunciados extra del TP3)**, usando la tabla de algoritmos de cálculo del monto final del enunciado **BASE** de ese trabajo (de nuevo: **SIN** incluir el algoritmo 4 que salía en el enunciado de los requerimientos extra de cada curso). Si el número identificador de algoritmo no está en la tabla (no es ni 1 ni 2 ni 3) entonces debe aplicarse el

cálculo normal del **TP1** (**NO** el cálculo del TP2). POR FAVOR, tengan a mano los enunciados de los TP anteriores, porque de otro modo no podrán avanzar en el desarrollo del TP4.

Una vez obtenido el monto final de un tratamiento, se debe determinar qué parte de ese monto cubre la obra social (**monto cubierto**) y qué parte queda a cargo del paciente (**monto no cubierto**). El porcentaje de cobertura depende del nivel de cobertura del afiliado y de la complejidad del tratamiento, según la siguiente tabla:

Tabla de porcentajes de cobertura

Nivel	Tratamiento regular ("R")	Tratamiento de alta complejidad ("A")
1	50%	40%
2	70%	60%
3	90%	80%

Para cada tratamiento se aplican las siguientes reglas:

a. Si el DNI del paciente **no se encuentra** en el padrón de afiliados, el monto cubierto es 0 y la totalidad del monto final es monto no cubierto (a cargo del paciente).
b. Si el DNI se encuentra en el padrón, se calcula el monto cubierto aplicando sobre el monto final el porcentaje que corresponda según la tabla anterior.
c. Si el monto cubierto así calculado es **mayor** al tope por atención del afiliado, entonces el monto cubierto se limita a ese tope (diremos que en ese tratamiento **se aplicó el tope**).
d. El monto no cubierto (a cargo del paciente) es la diferencia entre el monto final y el monto cubierto.

De esta forma, **cada tratamiento** tendrá su propio monto final, monto cubierto y monto no cubierto, que no dependen de los demás tratamientos del mismo paciente.

Ejemplo: supongamos un afiliado con nivel de cobertura 2 y tope por atención 60000, y un paciente que no figura en el padrón:

#	Paciente	Compl.	Monto final	Monto cubierto	Monto no cubierto
1	Afiliado	R	100000	60000 (el 70% sería 70000, pero el tope es 60000)	40000
2	Afiliado	A	80000	48000 (60%)	32000
3	No afiliado	R	50000	0	50000

d.) Requerimientos

Se solicita desarrollar un programa en Python, controlado por menú de opciones, que gestione los datos de ambos archivos. Las opciones del menú se definen como sigue:

Opción 1: Cargar Afiliados

Debe procesar el archivo "coberturas.csv" (línea por línea) y cargar cada uno de los afiliados en un vector de registros (u objetos), de tal manera que el vector se mantenga **siempre ordenado por DNI**, de forma ascendente. Para lograrlo, cada afiliado debe ser agregado al vector usando el algoritmo de **inserción ordenada** visto en clase (NO se permite cargar todo el vector y ordenarlo después, ni usar funciones nativas de ordenamiento). Si esta opción se ejecuta más de una vez, el vector debe volver a generarse desde cero.

Opción 2: Cargar Tratamientos

Debe procesar el archivo "tratamientos.csv" (línea por línea) y cargar cada tratamiento en un vector de registros (u objetos), en el mismo orden en el que aparecen en el archivo (exactamente igual que en la opción 1 del TP3). Si esta opción se ejecuta más de una vez, el vector de tratamientos debe volver a generarse desde cero.

Opción 3: Mostrar Resultados

Debe recorrer el vector de tratamientos generado en la opción 2 y, para cada tratamiento, calcular su monto final, buscar el DNI del paciente en el vector de afiliados generado en la opción 1 y aplicar las reglas de cobertura descriptas en la sección c.), para obtener los resultados que se piden a continuación. La búsqueda del DNI en el vector de afiliados debe realizarse obligatoriamente con el algoritmo de **búsqueda binaria** (aprovechando que el vector está ordenado por DNI).

Si la opción 3 se elige sin haber cargado previamente los afiliados (opción 1) y los tratamientos (opción 2), el programa debe mostrar un mensaje de error y no mostrar ningún resultado.

Resultados esperados (Opción 3):

- **r3.1:** Monto total cubierto por las obras sociales (sumando todos los tratamientos).
- **r3.2:** Monto total no cubierto, es decir, a cargo de los pacientes (sumando todos los tratamientos, incluidos los de pacientes que no son afiliados).
- **r3.3:** Cantidad de tratamientos en los que se aplicó el tope por atención (según la regla c. de la sección c.).
- **r3.4:** Cantidad de tratamientos de pacientes cuyo DNI no figura en el padrón de afiliados.
- **r3.5:** El código de la obra social que cubrió el mayor monto total (sumando lo cubierto en todos los tratamientos de sus afiliados). En caso de empate, informar el de menor código.
- **r3.6:** El DNI del paciente del tratamiento con mayor monto no cubierto. En caso de empate, informar el primero que se encuentre.

Los resultados r3.1 y r3.2 deben redondearse a 2 decimales usando la función round() (por ejemplo: round(monto, 2)).

Las opciones 1 y 2 pueden (opcionalmente) mostrar en pantalla algún mensaje informativo (por ejemplo, la cantidad de registros procesados), pero los resultados que serán evaluados son **exclusivamente** los que se muestren en la opción 3.

e.) Sobre las entradas y salidas

Se mantienen las consideraciones del enunciado anterior (TP3). En definitiva, el programa DEBE respetar lo siguiente:

1. Los archivos csv que se proveen como entrada se llaman "coberturas.csv" y "tratamientos.csv". Cuando usen la función open() para abrir esos archivos, deben asegurarse que esos (y no otros) sean los nombres enviados a la función. Un nombre diferente provocará un error en la ejecución del runner, y penalización de puntos en la calificación.
2. Mínimamente, el programa debe contar con **dos módulos**: uno para la definición de las clases y sus métodos asociados, y otro para el programa principal (conteniendo al menú). Que se entienda: el archivo de código fuente que contiene al programa principal DEBE llamarse **principal.py**: otro nombre podría provocar un error en el runner.
3. Todo módulo que entreguen (que contenga un script ejecutable, incluido el módulo principal) DEBE tener control de ejecución mediante la variable `__name__`.
4. Cuando se le solicite al usuario que elija una opción de menú, la misma solicitud debe indicarse con la leyenda: "Ingrese opción: ". No importa la forma en la que se presenten las opciones, pero el pedido de opción debe respetar ese mensaje en la consola. Por ejemplo:

```
1. Cargar afiliados
2. Cargar tratamientos
3. Mostrar resultados
0. Salir

Ingrese opción:
```

5. La opción para salir debe ser la **opción 0**. Nuevamente, no importa el nombre con el que se muestre esta opción en el menú, cuando el usuario ingrese la opción 0, el programa debe terminar. No muestre un mensaje de cierre. No muestre un mensaje de despedida. Nada. Solo termine.
6. Los **resultados** deben **mostrarse en el orden solicitado**, y utilizando los dos puntos (":") para separar el nombre del resultado, del valor del resultado en sí. Si, por ejemplo, el programa calcula r3.1 = 412350.25, r3.2 = 380122.8, r3.3 = 2, r3.4 = 1, r3.5 = 18 y r3.6 = 24984372, entonces la salida (cuando se seleccione la opción 3) debe ser:

```
r3.1: 412350.25
r3.2: 380122.8
r3.3: 2
r3.4: 1
r3.5: 18
r3.6: 24984372
```

7. **NO PUEDEN** usar en este TP4 estructuras de datos que no hayan sido tratadas en clase o en las fichas que abarca este trabajo. No usen diccionarios. No usen conjuntos. Usen una lista/arreglo de registros/objetos. La carga del vector de afiliados DEBE hacerse por **inserción ordenada** programada por ustedes, y la búsqueda de cada DNI DEBE hacerse por **búsqueda binaria** programada por ustedes (no usar funciones nativas como sort(), sorted(), index(), el operador in para buscar en una lista, ni el módulo bisect).

8. DEBEN usar **funciones** correctamente planteadas en cada lugar donde sea oportuno y aplicable. No es opcional. ES OBLIGATORIO USAR FUNCIONES EN TODA SITUACIÓN POSIBLE. No dispongan de un script con instrucciones sueltas. El arranque del programa DEBE estar contenido en una función principal, que luego invoque a otras funciones.